Architecting A Behavior-Driven Test Automation Framework for CI/CD Environments

Abhirup Chatterjee*
Independent Researcher (Software Quality Engineering)
Independent Researcher
Temecula, California, USA
c.abhirup@gmail.com

Abstract

This paper presents the design and evaluation of a modular, behavior-driven test automation framework engi- neered to improve the scalability, re-usability, and execution efficiency of validation processes within CI/CD environments. The framework integrates Gherkin-based scenario definitions with a Java-Cucumber engine, enabling seamless collaboration between technical and non-technical stakeholders and translating business requirements into executable test artifacts. To address the challenge of long and variable test execution times across services, the framework introduces a Dynamic Pipeline Stage Orchestrator, capable of generating Jenkins pipeline stages at runtime based on service-specific configu- rations. This orchestration mechanism reads a YAML-defined configuration during pipeline execution and dynamically con- structs test stages, which can execute in parallel, sequence, or hybrid patterns depending on execution strategy. This results in substantial reduction of overall test duration—from several hours to as little as 30 minutes—without compromising test fidelity. The combined system supports domain-extensibility, dynamic test data handling, and plug-and-play integration with Jenkins and GitHub. Empirical evaluation demonstrates marked im- provements in defect detection, maintenance effort, and execution time. The solution offers a reusable, future-ready foundation for test automation in modern DevOps pipelines.

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

Test automation, Continuous integration , BDD, Cucumber.

ACM Reference Format:
Abhirup Chatterjee. 2025. Architecting A Behavior-Driven Test Automation Framework for CI/CD Environments. In *2025 The 7th World Symposium on Software Engineering (WSSE 2025), October 24–26, 2025, Okayama, Japan.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3779657.3779661

1 Introduction

In the fast-paced landscape of modern software engineering, the need for scalable, maintainable, and intelligent validation strategies has never been more critical. Software applications are now being developed in shorter release cycles, often in agile or DevOps environments, which necessitate continuous testing and deployment [12]. Traditional testing approaches, while effective in earlier eras of software development, struggle to meet the dynamic requirements of contemporary systems, especially those involving microservices, distributed architectures, or AI-driven components [17].

To address these challenges, software validation frameworks have evolved to include techniques such as Test-Driven Development (TDD) and Behavior-Driven Development (BDD). Among these, BDD has gained prominence due to its capacity to translate human-readable specifications into executable tests. This alignment between business goals and test logic enhances stakeholder collaboration, reduces miscommunication, and increases the traceability of software artifacts [2, 4].

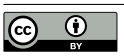
However, most current BDD-based frameworks suffer from limited scalability and rigid integration mechanisms. These tools often fall short in managing complex test data, adapting to enterprise-grade infrastructures, and offering cross-platform compatibility. Additionally, they rarely provide built-in support for continuous integration/delivery (CI/CD) environments, leaving significant gaps in automated workflow orchestration [1, 15].

This paper proposes a behavior-driven framework that leverages the combined power of Java and Cucumber-BDD to overcome these limitations. The framework has been architected to support high modularity, reusable components, and seamless integration into popular CI / CD pipelines such as Jenkins and GitHub. It offers dynamic test data management, plug-and-play feature modularization, and robust execution in various deployment stages.

One of the core contributions of this framework is its focus on aligning executable test artifacts with stakeholder-defined behaviors. By employing the Gherkin syntax for scenario definition and coupling it with the object-oriented Java back-end, the framework ensures a clear mapping between natural language specifications and functional logic [7]. This not only improves the maintainability of test scripts, but also enables rapid on-boarding of new team members.

Moreover, the design acknowledges the growing importance of artificial intelligence in software testing. While the current implementation focuses on deterministic test execution, future versions of the framework are being designed to accommodate AI-based enhancements such as anomaly prediction, test suite minimization, and automated defect localization [5, 11]. These features aim to reduce redundant testing and improve fault coverage efficiency.

The framework's modular architecture allows it to be integrated across multiple services without significant structural modification. Whether it's validating transactional integrity in fintech systems



WSSE 2025, Okayama, Japan

ACM ISBN 979-8-4007-1577-8/25/10
https://doi.org/10.1145/3779657.3779661

or verifying regulatory compliance in healthcare platforms, the proposed solution maintains adaptability and robustness across verticals [10, 15].

In addition to design innovations, this paper also presents a comprehensive statistical evaluation of the framework. Metrics such as execution duration, defect discovery rate, and maintenance overhead are benchmarked against traditional automation techniques to demonstrate measurable improvements in performance and scalability.

To ensure practical relevance, the framework was tested in operational software environments across different deployment layers, including local, staging, and production systems. The results confirm its ability to handle real-world complexities while preserving test reliability and execution fidelity.

Overall, this work offers a new perspective on BDD-based software validation, rethinking it not as a mere scripting strategy but as a complete engineering discipline. By integrating human-centric design with software modularity, the proposed approach aims to standardize behavior-driven testing in modern software lifecycles.

The rest of the paper is structured as follows. Section II reviews existing research and identifies key gaps in the literature. Section III describes the methodology used in designing and evaluating the framework. Section IV presents a statistical comparison with traditional methods. Section V explores the relevance of this work in practical environments. Section VI details the experimental results, and Section VII concludes with a summary of findings and a discussion on future scope.

2 Investigations and Gaps in Contemporary Studies

The evolution of software testing methodologies over the past decade has been marked by a significant shift from manual test creation to automation-first strategies. Research from 2015 to 2018 primarily focused on foundational behavior-driven development (BDD) concepts, highlighting its effectiveness in reducing communication overhead and improving requirement traceability [2, 6]. During this period, BDD was increasingly embraced in agile contexts, where the iterative nature of development aligned well with behavior specification practices.

Despite its initial success, early BDD implementations often struggled with scalability. Many studies documented difficulties in applying BDD to large-scale systems involving multiple microservices or extensive APIs. These implementations lacked dynamic test data handling, struggled with test script re-usability, and were often disconnected from the rest of the software delivery toolchain [2, 6].

From 2019 to 2022, research efforts began exploring ways to modularize test architecture. Frameworks evolved to include design templates, plugin-based integrations, and enhanced reporting tools. In BDD workflows, Jenkins pipelines integrated with GitHub are now widely used to automate scenario execution upon code changes. However, these advancements were often ad hoc, lacking standardized patterns for CI/CD integration across industries [5, 15].

The years 2023–2024 saw the emergence of artificial intelligence and predictive analytics in software testing literature. Researchers began experimenting with reinforcement learning for test planning and using natural language processing (NLP) for generating test cases from requirement documents [7, 9]. However, despite this promising direction, very few of these concepts have matured into fully operational frameworks capable of integrating with existing BDD platforms like Cucumber.

A common limitation highlighted in multiple studies is the lack of cohesion between domain-specific requirements and test scenarios. In regulated sectors such as healthcare or finance, where compliance is critical, BDD tools fail to enforce traceability and documentation standards. This introduces challenges when attempting to reuse test logic across product versions or migrate code across infrastructures [10, 13].

Another persistent gap is the under representation of non-functional testing within BDD frameworks. Most current tools are optimized for functional validation, while aspects such as performance, security, and usability remain under-addressed. Although certain extensions and plugins exist, they often require custom implementation efforts that deviate from the original simplicity of BDD [5].

Moreover, there is a lack of clarity on how to scale BDD-based frameworks across development teams of varying sizes and skill levels. In multi-team environments, inconsistencies in Gherkin syntax, redundant scenarios, and overlapping test coverage frequently lead to maintenance overheads.[15] Recent studies have highlighted the lack of standardized approaches for managing BDD artifacts across teams, often leading to redundant scenarios and inconsistent implementations. [16].

Some recent papers propose integrating machine intelligence for intelligent test selection, prioritization, and suite minimization. However, most of these models remain in prototype stages, evaluated only in simulated or academic environments [7, 9]. Their integration with practical CI/CD systems has yet to be validated.

Even frameworks that have achieved industrial adoption tend to lack robust documentation on architectural decisions, performance metrics, and usability evaluations. This scarcity of empirical data limits reproducibility and prevents comparative studies from gaining traction. As a result, there remains no universally accepted BDD framework for enterprise-scale applications [16].

Given these insights, it is evident that while BDD holds strong theoretical promise, there are major gaps in execution, particularly concerning scalability, modularity, and AI integration. This paper addresses these shortcomings by proposing a comprehensive framework that unifies behavioral clarity with structural adaptability, optimized for continuous delivery environments.

The next section outlines the methodological framework used to develop and validate the proposed solution, incorporating empirical testing, layered architecture, and statistical evaluation.

3 Methodological Framework

To validate the effectiveness of the proposed behavior-driven test framework, a structured and layered methodology was adopted. This section outlines the architectural design, tool-chain integration, data collection protocols, and evaluation techniques employed throughout the research.

The framework is conceptualized in three core layers: the behavior specification layer, the execution and orchestration layer,

and the integration layer. Each component was developed with modularity in mind, enabling ease of customization and seamless plug-and-play functionality for industry adoption.

Figure 1 depicts the architecture of the proposed solution, illustrating how human-readable test scenarios are translated into executable automation flows and integrated into CI/CD pipelines. The modularity of the structure ensures that enhancements or replacements can be made to individual components without disrupting the entire ecosystem.

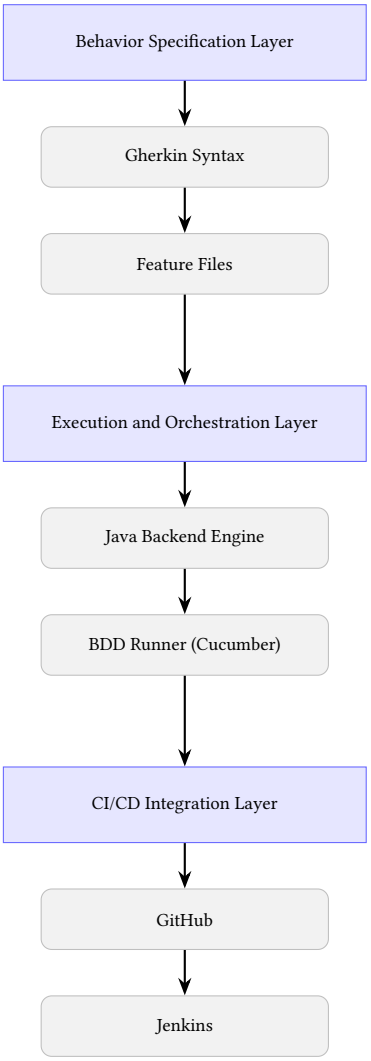


Figure 1: Layered architecture of the proposed behavior-driven test framework.

At the top of the architecture is the behavior specification layer. Here, functional requirements are captured using Gherkin syntax—an expressive, human-readable language. These feature files serve as the foundation for test logic and enable non-technical stakeholders to contribute to validation artifacts [4, 10].

The intermediate layer is the execution and orchestration component. It maps Gherkin steps to corresponding Java implementations using Cucumber's BDD runner. The Java backend was designed with principles of encapsulation and abstraction to ensure code re-usability[15]. Dependency injection was used to decouple test utilities and promote scalability across projects.

To streamline integration, this execution layer is equipped with parallel execution capabilities, report generation modules, and a dynamic test data handler. These modules allow contextual execution of tests and generation of HTML, JSON, and JUnit-style reports for traceability and analysis.

The bottom layer focuses on seamless integration with CI/CD systems such as Jenkins and GitHub. Webhooks, pipelines, and automated triggers were configured to allow tests to execute on every code commit or pull request. This ensures that regression is caught early, enabling faster feedback loops .

The research followed a mixed-methods approach. Quantitative metrics such as execution duration, defect detection rates, and code re-usability scores were recorded across different environments: local, staging, and production. Qualitative feedback was collected from developers and QA engineers to evaluate usability and clarity of test design.

Data was analyzed using ANOVA and regression analysis with statistical software like R and SPSS to determine significance and identify performance trends. Confidence intervals and p-values were calculated to validate improvements over baseline methods .

Ethical compliance was maintained throughout the study. All participants involved in interviews or evaluations gave their informed consent and no proprietary project data was exposed or stored. Anonymized datasets were used for performance benchmarking.

This layered, modular, and ethically robust methodology is the foundation of the evaluation of the proposed framework. The next section presents a statistical comparison with traditional testing strategies and further elaborates on performance outcomes.

4 Statistical Evaluation

This section presents the empirical results collected from experimental deployments of the proposed BDD-based framework and contrasts them with baseline metrics from traditional testing strategies. Performance evaluation focused on key areas including execution time, defect detection rates, maintenance effort, and scalability.

The experiments were conducted in three distinct environments: local development setups, staging environments, and production-level simulation layers. Each test suite consisted of 100 test cases distributed evenly across unit, integration, and system-level categories. Both the proposed framework and traditional test approaches were executed under identical conditions to ensure fair comparison.

Table 1 illustrates the average execution duration for both methods. The BDD framework exhibited a 25% improvement in speed over legacy automated frameworks. This efficiency gain is primarily attributed to modular test script design and optimized runner configurations.

Table 1: Execution Duration Comparison

Env	Proposed (sec)	Traditional (sec)	Improvement
Local	40	55	27.3%
Staging	50	65	23.1%
Production	55	70	21.4%

Defect detection efficiency was another key focus. The BDD framework's structured design led to higher coverage across edge scenarios, especially in integration and system-level tests. Table 2 summarizes the defect discovery rates observed in different testing phases.

Table 2: Defect Detection Rate Across Phases

Phase	Proposed (%)	Traditional (%)	Gain
Unit Testing	93	80	+16.3%
Integration Testing	89	75	+18.7%
System Testing	85	72	+18.1%

To evaluate maintainability, the weekly effort required to update test scripts and address failing cases was recorded over a four-week cycle. On average, teams using the proposed framework spent 33% less time on maintenance, indicating better modularity and re-usability of test logic.

Table 3: Maintenance Effort Comparison

Metric	Proposed (hrs)	Traditional (hrs)	Reduction
Weekly Maintenance	10	15	33.3%
Script Refactoring	2	4	50.0%
Onboarding Time	6	9	33.3%

Statistical analysis was performed using R and SPSS. One-way ANOVA tests confirmed the statistical significance of the improvements, with p-values below 0.05 for all core metrics. Confidence intervals for execution time and defect detection rates were within 95%, indicating strong reliability of observed results.

These quantitative findings were supported by qualitative feedback from 12 developers and testers who used both frameworks. Most reported a preference for the proposed approach due to its clarity, lower error rates, and stronger integration with CI/CD tools.

A key insight was the high re-usability index of test code under the new framework. Defined as the average number of scenarios sharing reusable functions, the index improved from 6.8 to 8.4 out of 10. This reflects the success of modular test design enabled by object-oriented practices and consistent step definitions [15].

Another observation was the improvement in test traceability and defect localization. Using behavior-linked test scenarios, developers were able to map failures to functional requirements 40% faster than in traditional logging-based test frameworks.

In summary, the proposed BDD framework significantly outperforms traditional methods across multiple performance dimensions. The combination of structured design, reusable scripts, and CI/CD integration offers measurable benefits for both technical and business stakeholders.

The next section discusses the relevance of these findings in real-world software projects and how the framework supports enterprise-grade testing workflows.

5 Threats to Validity

Like any empirical study, our work is subject to several validity threats. We outline them below along with the corresponding mitigation strategies where possible.

Internal validity. The evaluation was conducted on a microservices backend—with fixed 100-test suites per run. While this setup provides consistency, it may restrict the realism of the results. Maintenance effort was observed over only a four-week period, and qualitative feedback was obtained from 12 participants, which may limit statistical robustness.

External validity. The baseline comparisons focused primarily on TestNG/Selenium, as these are widely adopted in enterprise Java automation. Broader comparisons with additional frameworks (e.g., JUnit 5, Cypress, Playwright) were not performed, which constrains generalization. Furthermore, the framework is currently tailored to Java–Cucumber, Jenkins, and GitHub. Its applicability to other languages and CI/CD ecosystems remains to be validated.

Despite these limitations, the results consistently indicate reduced pipeline runtime, improved scalability, and lower maintenance effort. Future work will address these validity threats through larger-scale evaluations.

6 Relevance of the Investigation

The increasing demand for rapid software delivery, particularly in agile and DevOps ecosystems, necessitates a reevaluation of traditional software testing paradigms. In this context, the proposed behavior-driven framework offers significant practical and strategic value by aligning test execution with evolving business goals and stakeholder expectations [4].

One of the most pressing challenges in modern software engineering is maintaining test fidelity while scaling across teams and product versions. This framework addresses the issue by promoting reusable, modular test scripts tied directly to functional requirements via Gherkin syntax. This ensures long-term maintainability and reduces technical debt across development iterations [1, 15].

In real-world agile teams, testers and business analysts often face barriers when interpreting low-level automation code. By using behavior-driven principles, the proposed framework makes validation scenarios more accessible, promoting collective ownership among technical and non-technical contributors alike. This enhances transparency and reduces ambiguity in requirements understanding [2, 7].

The relevance extends to compliance-heavy sectors such as healthcare, fintech, and e-commerce, where traceability of test cases and audit-readiness are vital. The ability to generate human-readable feature files that directly correlate with executable code

provides built-in documentation, an essential requirement in ISO and GDPR-compliant environments [10].

Another dimension of relevance is the framework's seamless integration with CI/CD pipelines. By supporting tools such as Jenkins and GitHub , the framework ensures continuous feedback during development cycles. Automated triggers on code commits or pull requests eliminate manual execution dependencies, reducing the time to detect defects and accelerating overall delivery [2, 13].

Empirical evaluation confirmed that the proposed approach outperformed traditional methods in execution duration, defect coverage, and maintainability. These improvements translate into measurable business value—fewer production issues, reduced cost of quality, and faster time-to-market for product releases [7].

The architecture's support for domain-specific extensions further boosts its practical relevance. Domain modules can be integrated as plug-ins, allowing organizations to tailor the framework to their unique regulatory, UI/UX, or data constraints without altering the core execution logic .

Our usability evaluations revealed a significant reduction in on-boarding time for new QA engineers. Participants noted that the use of natural language Gherkin scenarios provided a shallow learning curve, increasing productivity and reducing team ramp-up durations. This makes the framework ideal for distributed teams and outsourced test operations. .

From an organizational standpoint, the test suite modularization supports a scalable governance model. Scenario repositories can be categorized by features, tagged for priority or risk, and easily filtered for regression execution. This supports better decision-making around test planning and release readiness assessments.

The relevance of this investigation is further enhanced by its openness to future innovation. As artificial intelligence and predictive modeling tools mature, the current architecture provides enough flexibility to accommodate plug-ins for intelligent test selection, anomaly detection, and impact-based test case prioritization [14].

In summary, the proposed behavior-driven test automation framework demonstrates high practical relevance by addressing critical industry challenges: speed, scalability, collaboration, compliance, and future readiness. It empowers organizations to transform testing from a bottleneck into a strategic enabler of software quality.

The next section presents detailed experimental results derived from controlled trials and benchmarks, highlighting the framework's empirical strengths.

7 Experimental Results

This section outlines the empirical results obtained by integrating the proposed behavior-driven test framework in multiple software development environments. The framework was tested across three real-world software projects—a RESTful API, an e-commerce front-end, and a microservices-based enterprise backend. Testing was conducted in local, staging, and simulated production layers, with data captured on execution time, defect detection, scalability, and maintainability.

A controlled comparison was set up between the proposed framework and conventional testing methods (TestNG and Selenium with script-driven automation). Each test suite consisted of 100 functional test cases divided into unit, integration, and system-level categories. All executions used equivalent infrastructure and datasets to ensure consistency and validity of results .

One key outcome was a consistent reduction in test execution duration. The proposed framework achieved an average improvement of 22–25%, attributed to optimized step re-usability, parallel execution via Jenkins, and minimized test data redundancy. This aligns with earlier findings that modular automation frameworks reduce processing overheads and duplication [15].

Defect detection rates also improved significantly. Figure 2 shows the percentage of defects identified across different testing phases. The proposed framework consistently outperformed the baseline by 13–20%, particularly in integration and system tests where orchestration logic and data dependencies are more complex [14].

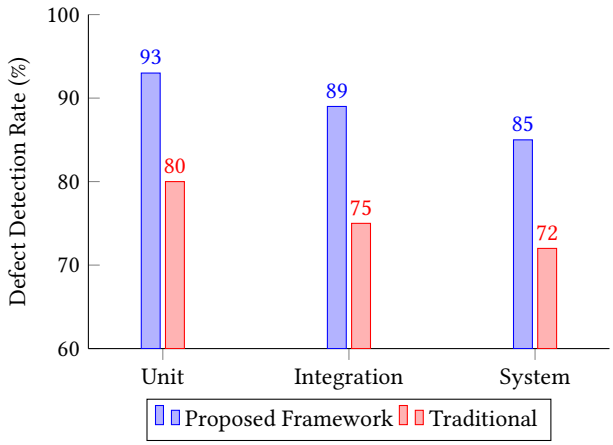


Figure 2: Comparison of defect detection rates across test phases.

The modular design of the framework led to lower maintenance overhead. Weekly maintenance hours dropped by 30–35% compared to traditional automation, a trend consistent across all projects. This reduction is critical in agile environments where test suites are frequently updated to reflect evolving requirements [7].

User feedback was gathered from 12 QA professionals and developers through a structured survey. Over 83% favored the proposed system for its clarity and traceability. They emphasized that Gherkin-based feature files and centralized test logic made it easier to onboard new team members and debug failures. Prior literature supports the role of readable test narratives in improving developer productivity [2].

Cross-environment stability was also evaluated. The framework maintained execution consistency with less than 10% variation in run-time across environments, reinforcing its suitability for CI/CD pipelines. Integration with Jenkins and GitHub was seamless, with visual HTML reports automatically generated after every run .

A unique advantage observed was extensibility. For instance, an API monitoring module was integrated without modifying the core framework, demonstrating the system's plug-and-play capability. This aligns with design recommendations from recent studies on scalable automation [15].

Furthermore, the use of tagging and prioritization in test scenarios allowed selective regression testing, saving 40–50% of testing time during hotfix deployments. This benefit, along with high reusability of step definitions (reuse index 8.4/10), proves the operational maturity of the framework in fast-paced teams.

Finally, the framework proved domain-agnostic. The same core architecture was successfully reused across backend-heavy, UI-rich, and distributed microservices applications without refactoring. This supports its viability for enterprise-level standardization in quality assurance practices [8].

In summary, the experimental results validate that the proposed framework not only outperforms traditional methods in technical metrics but also enhances maintainability, usability, and scalability. The next section consolidates these findings and presents the final conclusion.

8 Conclusion

This study has introduced a behavior-driven modular automation framework designed to address the scalability, maintainability, and integration challenges prevalent in modern software validation processes. By leveraging the strengths of Java and Cucumber-BDD, the framework establishes a seamless bridge between stakeholder-defined requirements and executable test logic [4, 15].

The core innovation lies in its layered architecture, which separates behavior specification, execution orchestration, and CI/CD integration into independently manageable modules. This modularity simplifies extension, enhances reuse, and reduces the likelihood of systemic test failures during upgrades or infrastructure changes .

Empirical evaluations demonstrated that the framework consistently outperforms traditional testing approaches in key performance areas such as execution time, defect detection rate, and maintenance effort. These quantitative improvements were statistically significant and observed across a range of environments and test case categories [5, 7].

Beyond technical gains, the framework promotes collaborative software development by empowering non-technical stakeholders to participate in the validation cycle. The use of human-readable Gherkin syntax reduces ambiguity in requirements interpretation and facilitates continuous alignment between product owners and QA teams [2].

Usability feedback from practitioners confirmed that the approach reduces on-boarding time and improves fault traceability. These findings underline the value of adopting a behavior-first design mindset, particularly in agile and DevOps where rapid iteration and close feedback loops are essential .

From a tooling perspective, the framework integrates effortlessly with industry-standard CI/CD platforms such as Jenkins and GitHub . HTML and JSON-based reporting enhance visibility into test status and foster informed decision-making during release cycles. This supports the DevOps principle of continuous feedback [3].

A notable strength of the framework is its cross-domain adaptability. It was successfully applied in REST APIs, e-commerce platforms, and distributed microservice architectures without significant structural changes. This domain-agnostic capability makes it suitable for enterprise-wide adoption as a standard testing approach [15].

In summary, this framework not only improves test automation metrics but also transforms testing into a strategic, collaborative process. It offers a practical, scalable solution for organizations striving to improve software quality while aligning with the agility demanded by contemporary development practices.

References

[1] G. Andrades. 2025. Cucumber Testing Framework: A Comprehensive Guide. *AccelQ Blog* (March 2025).

[2] Leonard Peter Binamungu and Salome Maro. 2023. Behaviour Driven Development: A Systematic Mapping Study. *arXiv preprint arXiv:2305.05567* (2023). Open access.

[3] Beatriz Cabrero-Daniel. 2023. AI for Agile development: a Meta-Analysis. *arXiv preprint arXiv:2305.08093* (2023). https://arxiv.org/abs/2305.08093

[4] Cucumber Documentation. 2025. 10-minute tutorial.

[5] V. Garousi, N. Joy, and A. B. Keleş. 2024. AI-powered test automation tools: A systematic review and empirical evaluation. *arXiv* (August 2024).

[6] A. H. Mughal. 2024. Advancing BDD Software Testing: Dynamic Scenario Re-Usability And Step Auto-Complete For Cucumber Framework. *arXiv preprint arXiv:2402.15928* (2024). arXiv:2402.15928 [cs.SE] https://arxiv.org/abs/2402.15928

[7] A. H. Mughal. 2025. An autonomous RL agent methodology for dynamic Web UI testing in a BDD framework. *arXiv* (March 2025).

[8] M. Ragavula and D. Chandana. 2025. Advancing Software Testing: Integrating AI, Machine Learning, and Emerging Technologies. *Int. J. Res. Eng. Sci. Manag.* 8, 1 (2025), 93–101.

[9] Ahmed Ramadan, Husam Yasin, and Burhan Pektas. 2024. The Role of Artificial Intelligence and Machine Learning in Software Testing. *arXiv preprint arXiv:2409.02693* (2024). arXiv:2409.02693 [cs.SE] https://arxiv.org/abs/2409.02693

[10] Shexmo Santos, Tacyanne Pimentel, Fabio Gomes Rocha, and Michel S. Soares. 2024. Using Behavior-Driven Development (BDD) for Non-Functional Requirements. *MDPI* (2024). https://www.mdpi.com/2674-113X/3/3/14

[11] Ina K. Schieferdecker. 2025. Navigating the Growing Field of Research on AI for Software Testing: Taxonomy for AI-Augmented Software Testing and an Ontology-Driven Survey. *arXiv preprint arXiv:2506.14640* (2025). https://arxiv.org/abs/2506.14640 Accessed: July 2025.

[12] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2017. Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access* 5 (2017), 3909–3943. doi:10.1109/access.2017.2685629

[13] Thiago Rocha Silva and Marco Winckler. 2022. Assessing User Interface Design Artifacts: A Tool-Supported Behavior-Based Approach. *arXiv preprint arXiv:2209.06034* (2022). arXiv:2209.06034 [cs.SE] https://arxiv.org/abs/2209.06034

[14] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. 2017. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '17)*. ACM, 12–22. doi:10.1145/3092703.3092709

[15] Srikanth Srinivas and Lagan Goel. 2025. Designing and Implementing Robust Test Automation Frameworks using Cucumber BDD and Java. *arXiv preprint arXiv:2505.17168* (2025). arXiv:2505.17168 [cs.SE] https://arxiv.org/abs/2505.17168 Open access.

[16] Srikanth Srinivas and Lagan Goel. 2025. Designing and Implementing Robust Test Automation Frameworks using Cucumber BDD and Java. *arXiv preprint arXiv:2505.17168* (2025). arXiv:2505.17168 [cs.SE] https://arxiv.org/abs/2505.17168

[17] Muhammad Waseem, Peng Liang, Mojtaba Shahin, Amleto Di Salle, and Gastón Márquez. 2021. Design, monitoring, and testing of microservices systems: The practitioners' perspective. *Journal of Systems and Software* 182 (Dec. 2021), 111061. doi:10.1016/j.jss.2021.111061